L1 Engineer's Guide

Pacific Life — AppDynamics → OpenTelemetry / Grafana Cloud

Trace3 Delivery · every tool, where it runs, how it connects, where we are, what done looks like

L1 Engineer's Guide — Pacific Life AppDynamics → OpenTelemetry/Grafana

> Read this first if you are new to the project. It explains every tool, where it runs, who owns it, how the pieces connect, where we are today, and what "done" looks like. Diagrams are plain text so they survive DOCX/email.

---

1. The one-paragraph version

Pacific Life pays for **AppDynamics (AppD)** to monitor 10 applications. The contract ends **31 Dec 2026** and they do not want to renew. Our job (Trace3) is to replace the AppD agent inside each app with an **OpenTelemetry (OTel)** agent, send that data through **Grafana Alloy** (a collector Pacific Life already runs) into **Grafana Cloud** (which Pacific Life already owns), rebuild the same alerts/dashboards there as code, prove the app teams can see the same things they saw before ("functional parity"), make sure alerts still create **ServiceNow** tickets in the right team's queue, and hand Pacific Life an evidence pack so they can safely let AppD expire.

---

2. Glossary — every tool and term, in plain words

Monitoring concepts

Term	Plain meaning
APM (Application Performance Monitoring)	Watching what happens *inside* an application: which requests came in, how long they took, which failed, which database/API calls they made.
Trace	The full story of one request as it travels through the app (and other services). Made of **spans**.
Span	One step inside a trace, e.g. "HTTP GET /api/quote took 120 ms" or "SQL SELECT took 15 ms". Has a name, start/end time, status (OK/ERROR), and attributes (key=value, like `http.route=/api/quote`).
Metrics	Numbers over time: requests per second, error %, CPU. Cheap to store, good for alerting.
Logs	Text lines the app writes. Useful when correlated to a trace via `trace_id`.
RED	**R**ate (calls/min), **E**rrors (%), **D**uration (latency, e.g. p95). The three golden numbers per service/endpoint.
p95	95% of requests were faster than this value. Better than "average" because it shows the slow tail.
Instrumentation	The code that produces traces/metrics/logs. **Zero-code / auto-instrumentation** = an agent injected at startup does it for you; no app code changes.
Dual-run	Two monitoring agents in the same process at the same time (old + new) so you can compare. Possible for Java and Python; **impossible for .NET**.

Term	Plain meaning
Cardinality	Number of unique label combinations in a metric. Too many (e.g. a user ID in a span name) = huge cost. We guard against it.
Tail sampling	Deciding *after* a trace completes whether to keep it (keep all errors, all slow ones, 10% of normal). Saves cost.
PII	Personal data (SSN, card numbers, emails). Must be scrubbed before it leaves Pacific Life's network.

AppDynamics terms (the thing we are replacing)

AppD term	Meaning	OTel/Grafana equivalent
Controller	The AppD server/UI where everything is stored and viewed. Stays up until Dec 31 as fallback.	Grafana Cloud
Agent	Library attached to the app (Java `-javaagent`, .NET CLR profiler, Python `pyagent`) that sends data to the Controller. **This is what we remove.**	OTel zero-code agent
Business Transaction (BT)	A named entry point into the app, e.g. "Get Quote", "Bind Policy". AppD auto-detects them and lets you add custom split rules (by URL segment, HTTP header, or Java class/method = "POJO").	A **span-metrics series** keyed on `service.name` + `span_name` (`http.route`). Custom split rules are rewritten as **OTTL** rules in the pipeline.
Health Rule	A condition on a BT or tier: "error rate > 2% for 5 min". Can be **static** (fixed number) or **dynamic baseline** (AppD learns "normal").	**Grafana alert rule** on span-metrics. Baseline rules need a decision: static threshold or Grafana ML.
Policy / Action	"When health rule X fires → create ServiceNow incident in group Y".	Grafana **notification policy + contact point** (webhook to ServiceNow).
Snapshot	A captured slow/error request with full detail.	A **trace in Tempo**, kept by tail sampling.
Dashboard	Visual panels.	Grafana dashboard (built from a Terraform template).
Information Point / Data Collector	Custom business values captured from code (e.g. "policy amount").	Span attribute → may need app code change (Pacific Life owns) or becomes an exception.
EUM / DB Visibility / Analytics / Synthetics	Browser monitoring, DB-server monitoring, business analytics, synthetic checks.	No zero-code equivalent — Pacific Life must decide in-scope or exception (Discovery Q2).

OpenTelemetry (OTel) terms (the new standard)

Term	Meaning
OpenTelemetry	Open-source, vendor-neutral standard for producing telemetry. Owned by CNCF, not a vendor. Every APM vendor now accepts it.
OTLP	OpenTelemetry Protocol — the wire format. gRPC on port **4317**, HTTP on port **4318**.
OTel Java agent	A `.jar` added with `-javaagent:` at JVM start. Auto-instruments Spring, Tomcat, JDBC, JMS, etc.
OTel .NET auto-instrumentation	A CLR profiler registered via `COR_PROFILER` env vars. Auto-instruments ASP.NET, HttpClient, SQL. **Only one profiler per process** — this is why AppD and OTel cannot coexist on .NET.
OTel Python distro	`opentelemetry-instrument your_app` wrapper that monkey-patches Django/Flask/FastAPI, requests, DB drivers. Weakest coverage of the three.
Resource attributes	Metadata about *who* is sending: `service.name`, `app.id`, `deployment.environment`. Set via `OTEL_RESOURCE_ATTRIBUTES`.

Term	Meaning
OTel Collector	A standalone process that receives, processes, and exports telemetry. Grafana Alloy *is* an OTel Collector distribution (plus more).
OTTL	OpenTelemetry Transformation Language — small statements run inside the collector to rename spans, set/delete attributes, redact. We use it to reproduce AppD BT rules and PII redaction. Example: `set(name, "BT:quote") where IsMatch(attributes["url.path"], "^/api/quote")`.

Grafana stack terms

Term	Meaning	Where it runs
Grafana Alloy	Grafana's collector. Reads OTLP, Prometheus, logs; processes; forwards. Config files end in `.alloy`.	Two places: **on every app host** (already deployed by Pacific Life) and as a **regional pipeline on EKS**.
Amazon EKS	AWS-managed Kubernetes. Hosts the regional Alloy pipeline (3 replicas, behind an internal NLB).	AWS, Pacific Life account
Grafana Cloud	Grafana's SaaS. Pacific Life owns the account and pays for ingest. Contains the four services below.	SaaS (Grafana-hosted)
Tempo	Trace storage/query. Has a **metrics-generator** that turns spans into RED metrics automatically.	Inside Grafana Cloud
Mimir	Prometheus-compatible metrics storage. Receives host metrics *and* the span-metrics Tempo generates. Alert rules query Mimir.	Inside Grafana Cloud
Loki	Log storage/query.	Inside Grafana Cloud
Span-metrics	Metrics Tempo generates from traces: `traces_spanmetrics_calls_total` (rate+errors) and `traces_spanmetrics_latency_bucket` (duration), labelled by `service`, `span_name`, `status_code`. **These are our BT equivalents.**	Mimir
Application Observability	Grafana Cloud UI that shows a service map and RED per service/operation from span-metrics. Nice-to-have; Q9 confirms it's on.	Grafana Cloud
Grafana Alerting	Alert rules (query + threshold), contact points (where to send), notification policies (routing by labels).	Grafana Cloud
TraceQL / PromQL / LogQL	Query languages for Tempo / Mimir / Loki.	

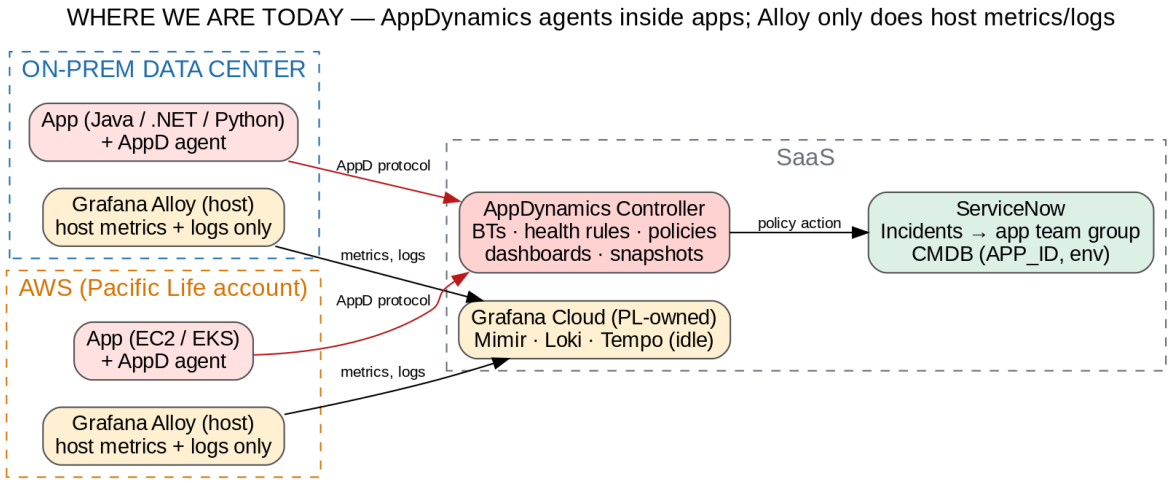
Delivery tooling

Term	Meaning
Terraform	Infrastructure-as-code. We describe dashboards, alert rules, contact points in `.tf` files; Terraform creates them in Grafana via the **Grafana provider**. Repeatable for all 10 apps and the 343 after.
Azure DevOps (ADO)	Pacific Life's Git + CI/CD. Our Terraform and Alloy configs live here; pipelines run `plan` / `apply`. State in Azure Storage.
Helm	Kubernetes package manager. Deploys the Alloy pipeline to EKS with `values.yaml`.
ServiceNow (SNOW)	Pacific Life's ITSM. Alerts become **incidents** assigned to an **assignment group** (a team queue). Also holds the **CMDB**.
CMDB	Configuration Management Database. Source of truth for `APP_ID` and environment per host/app.
CAB	Change Advisory Board. Approves production changes. Has a lead time we must plan around.

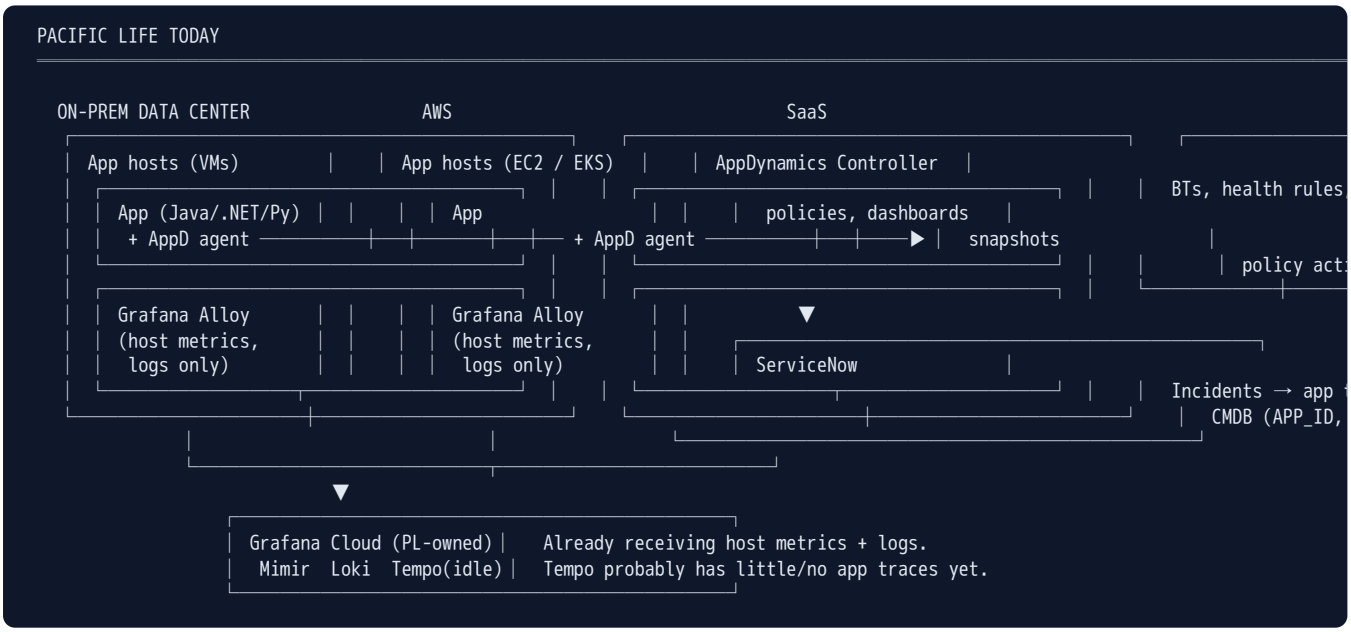
Term	Meaning
Hypercare	Fixed window (10 business days) after prod cutover when we watch closely and can roll back fast.
Change freeze	Dec 21 – Jan 3: no production changes. So Dec 18 is our real last change date.
FPC	Functional Parity Checklist — our per-app acceptance document. If it isn't ticked and signed, the app isn't done.

---

3. Where we are today (current state)



01 current state

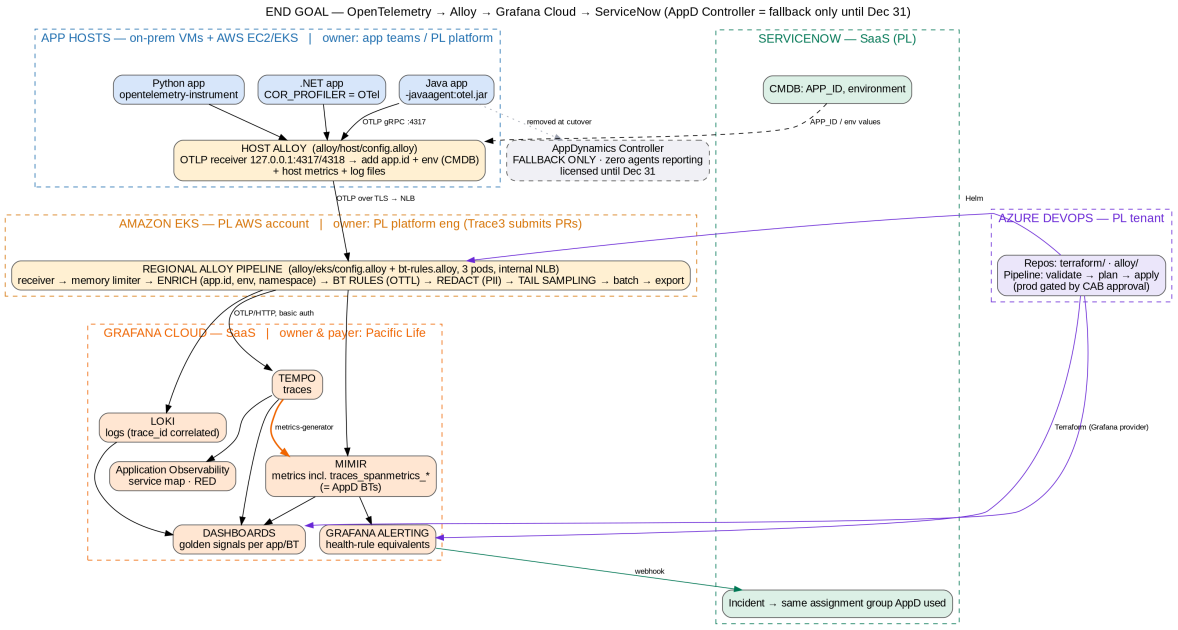


What exists already (we reuse): Alloy on every host; Grafana Cloud account; Azure DevOps; ServiceNow + CMDB with `APP_ID` tagging; an Alloy pipeline on EKS (owned by Pacific Life platform team).

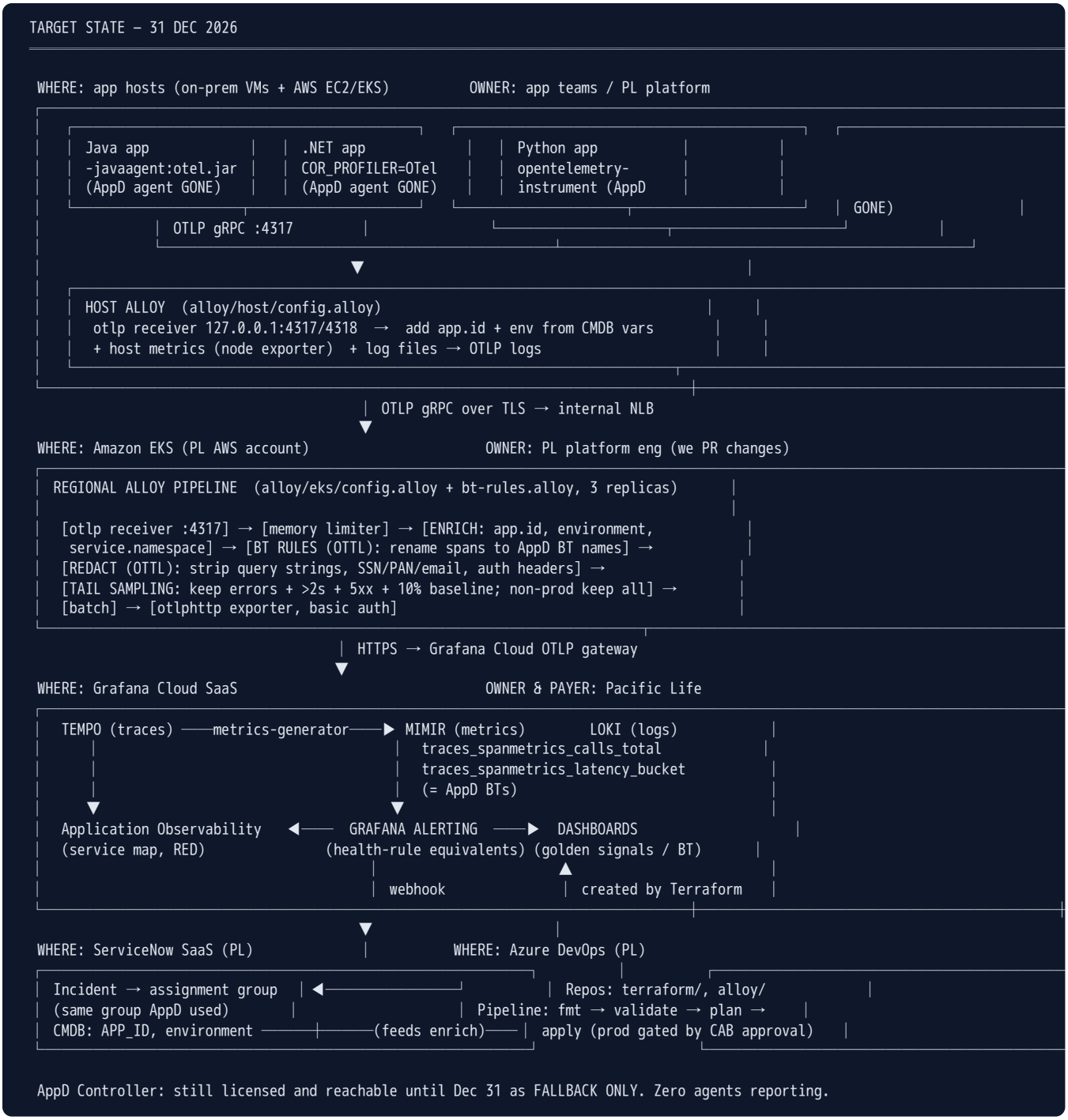
What does *not* exist yet: OTel agents in the apps; BT/PII rules in the pipeline; span-metrics; alert rules/dashboards in Grafana for these apps; the Grafana → ServiceNow routing per app; any parity evidence.

Project status: Phase 0. Discovery Request is drafted (21 questions), templates and reference configs are written, nothing has been changed on any Pacific Life system yet.

4. End goal (target state) — with hosting



02 target state



"Done" means, for each of the 10 apps: OTel agent on, AppD agent off; BTs visible as span-metrics within ±10% of AppD; every health rule has a Grafana alert; a test alert created a ServiceNow incident in the same group; dashboards exist; exceptions approved in writing; rollback rehearsed; owner signed the FPC. Then all 10 roll up into the **Decommission-Readiness Pack** (draft Nov 27, final Dec 18).

---

5. Follow one request end-to-end



03 request flow

Take `GET /api/quote/123` on APP-01 (Java) in prod after cutover.

```
1. Tomcat receives the request.
   OTel Java agent (loaded via -javaagent) creates a SERVER span:
     name = "GET /api/quote/{id}"   http.route="/api/quote/{id}"   url.path="/api/quote/123"
     resource: service.name="app01-quote-api"  app.id="APP-01"  deployment.environment="prod"
   Child spans: JDBC SELECT (CLIENT span), outbound HTTP to rating service (CLIENT span).
   Logback appender sends log lines with trace_id as OTLP logs.

2. Agent exports the finished trace over OTLP gRPC to 127.0.0.1:4317  → HOST ALLOY.

3. HOST ALLOY: adds host.name; inserts app.id/deployment.environment if agent didn't; batches;
   sends to the regional pipeline NLB (TLS).

4. EKS PIPELINE:
   enrich  → service.namespace="APP-01" (so Grafana groups services per app)
   bt_rules→ APP-01 rule: url.path matches ^/api/(?P<bt>[^/]+) → bt.name="quote"
             → span name becomes "BT:quote"; if header x-channel=mobile → "BT:quote@mobile"
            (this is exactly the AppD BT "Quote (mobile)")
   redact  → url.full loses "?..." ; any SSN-looking string → "[SSN]" ; SQL literals → "?"
   sampling→ waits 15s for the whole trace; keeps it if ERROR / >2s / 5xx / lucky 10%
   batch   → otlphttp → https://otlp-gateway-...grafana.net/otlp  (basic auth: instance id + token)

5. GRAFANA CLOUD:
   Tempo stores the trace (if sampled).
   Tempo metrics-generator increments:
     traces_spanmetrics_calls_total{service="app01-quote-api", span_name="BT:quote", status_code="STATUS_CODE_OK"}
     traces_spanmetrics_latency_bucket{... le="0.25"}
   → written into Mimir. Loki stores the log lines with trace_id.

6. ALERTING (every 60s):
   rule "APP-01 | BT:quote | error rate > 2%" runs PromQL against Mimir.
   If true for 5 min → alert fires with labels app_id=APP-01, severity=critical.
   Notification policy matches app_id=APP-01 → contact point "snow-app-01-prod" (webhook)
   → ServiceNow creates incident, assignment_group="PL-Quote-Platform-L2".

7. HUMAN: opens the APP-01 Golden Signals dashboard → sees BT:quote error % spike →
   clicks into Tempo "recent error traces" → opens the trace → sees the JDBC span failing →
   clicks the trace_id → Loki shows the stack trace log line.
```

That chain — **agent → host Alloy → EKS pipeline → Tempo/Mimir/Loki → alert → ServiceNow** — is the whole project. Every deliverable configures one link of it.

---

6. Hosting & ownership map

Component	Hosted where	Owned by	Paid by	Who changes it	Our repo artifact
Applications (10)	On-prem VMs, AWS EC2, some EKS	PL app teams	PL	App teams (via their CI/CD) — we provide the exact flags	`agents/java.md`, `dotnet.md`, `python.md`
OTel agents	Inside each app process	OSS (CNCF)	—	App teams deploy; we specify version	same
Host Alloy	Every app host	PL platform eng	PL	PL (already deployed); we may request receiver enablement	`alloy/host/config.alloy` (reference)

Component	Hosted where	Owned by	Paid by	Who changes it	Our repo artifact
Regional Alloy pipeline	Amazon EKS, PL AWS account, 3 pods behind internal NLB	PL platform eng	PL (AWS)	**We PR changes**; PL merges; CAB for prod	`alloy/eks/config.alloy`, `bt-rules.alloy`, `values.yaml`
Grafana Cloud (Tempo/Mimir/Loki/Alerting/Dashboards)	Grafana SaaS	PL	PL (ingest licence)	**We** via Terraform	`terraform/**`
Terraform + pipelines	Azure DevOps (PL tenant), state in Azure Storage	PL DevOps	PL	We author; PL approves prod	`azure-pipelines.yml`
ServiceNow + CMDB	SaaS (PL)	PL SNOW team	PL	PL creates webhook user/groups; we configure Grafana side	`terraform/modules/servicenow-routing`
AppDynamics Controller	SaaS or on-prem (PL)	PL	PL until Dec 31	Nobody — read-only fallback, export source	evidence folder

---

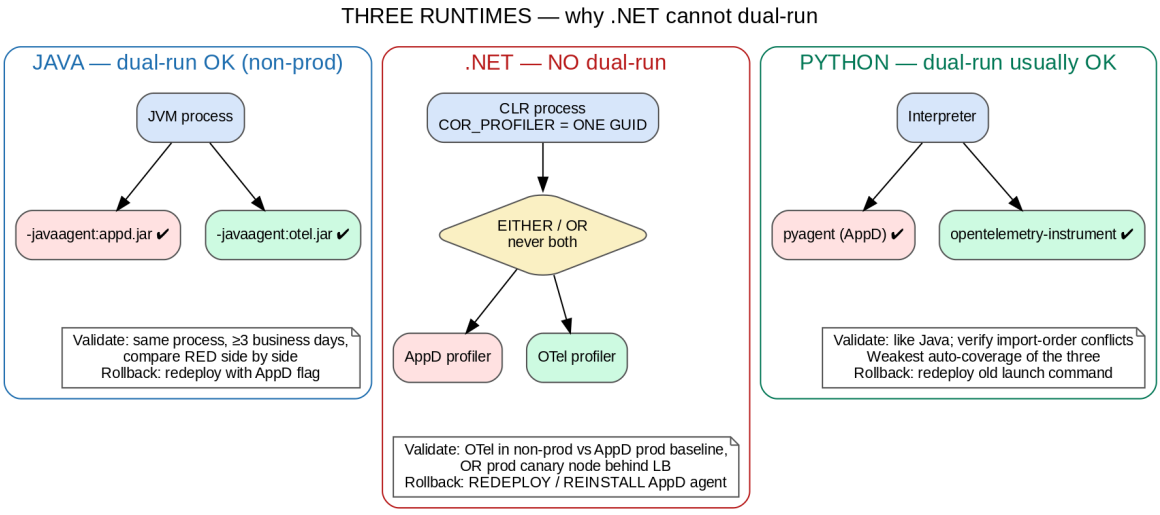
7. Integration points (what talks to what)

From → To	Protocol / port	Auth	Network path	Failure symptom
App agent → Host Alloy	OTLP gRPC `127.0.0.1:4317` (or HTTP 4318)	none (localhost)	loopback	No spans in Tempo; agent log "connection refused"
Host Alloy → EKS pipeline	OTLP gRPC :4317 over TLS	mTLS/TLS (PL standard)	on-prem → AWS (firewall rule needed), EC2 → NLB	Host Alloy log: exporter retries/queue full
EKS pipeline → Grafana Cloud	OTLP/HTTP over HTTPS (`otlp-gateway-<region>.grafana.net/otlp`)	Basic auth: instance ID + API token (K8s secret)	EKS egress → internet	401/429 in pipeline logs; gaps in Mimir
Tempo → Mimir	internal (metrics-generator)	—	inside Grafana Cloud	`traces_spanmetrics_*` absent → metrics-generator not enabled (Q9)
Grafana Alerting → ServiceNow	HTTPS webhook POST	Basic auth (integration user)	Grafana Cloud → SNOW SaaS	Contact point test fails; no incident

From → To	Protocol / port	Auth	Network path	Failure symptom
Azure DevOps → Grafana Cloud	HTTPS (Grafana API) via Terraform provider	Service-account token (variable group)	ADO agent → internet	`terraform apply` 401/403
Azure DevOps → EKS	`kubectl`/Helm via service connection	AWS IAM	ADO agent → EKS API	Helm upgrade fails
CMDB → pipeline	Not live — values copied into `OTEL_RESOURCE_ATTRIBUTES` / host env at provisioning	—	—	`app.id="UNMAPPED"` in traces
App → AppD Controller (today)	AppD proprietary over HTTPS	account key	—	Removed at cutover; Controller shows node "not reporting" = our proof

---

8. The three runtimes — and why .NET is different



04 runtimes

```
JAVA  (dual-run OK in non-prod)          .NET  (NO dual-run)              PYTHON (dual-run usually OK)
┌──────────────────────────┐            ┌──────────────────────┐         ┌──────────────────────────┐
│ JVM                      │            │ CLR                  │         │ Interpreter              │
│   -javaagent:appd.jar  ✔ │            │   COR_PROFILER = ONE GUID │    │   pyagent (AppD) ✔       │
│   -javaagent:otel.jar  ✔ │            │    either AppD  ──or── OTel │  │   opentelemetry-instrument ✔ │
│   both instrument bytecode │          │    never both        │         │   both monkey-patch      │
└──────────────────────────┘            └──────────────────────┘         └──────────────────────────┘

Validate: side-by-side in the SAME      Validate: OTel in non-prod vs AppD   Validate: like Java, but
process ≥3 business days, compare RED.   prod baseline, OR prod canary node.   verify import-order conflicts.
Rollback: redeploy with AppD flag.      Rollback: REDEPLOY/REINSTALL AppD.    Rollback: redeploy old command.
```

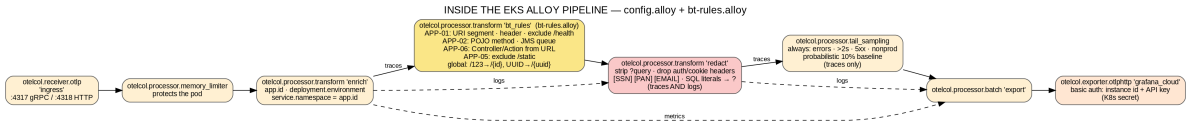
How each gets enabled (details in `agents/*.md`):

Runtime	Enable OTel	Remove AppD	Gotchas
Java	Add `-javaagent:/opt/otel/opentelemetry-javaagent.jar` + `OTEL_*` env vars; restart	Remove AppD `-javaagent` line	Other agents (security/profilers) may conflict; WebLogic/WebSphere need admin-console changes

Runtime	Enable OTel	Remove AppD	Gotchas
.NET	Install OTel auto-instrumentation PowerShell module; `Register-OpenTelemetryForIIS` ; `iisreset`	Uninstall AppD .NET agent MSI first	Framework < 4.6.2 unsupported; ASP.NET classic often lacks `http.route` → pipeline derives BT from URL
Python	`pip install opentelemetry-distro opentelemetry-exporter-otlp; opentelemetry-bootstrap -a install` ; wrap with `opentelemetry-instrument`	Drop `pyagent` wrapper / package	gunicorn `--preload` needs `post_fork` hook; unsupported frameworks → code change

---

9. Inside the EKS pipeline (what each stage does and why)



05 pipeline stages

```
config.alloy                                          bt-rules.alloy
──────────────────────────────────────────────────────────────────────────────

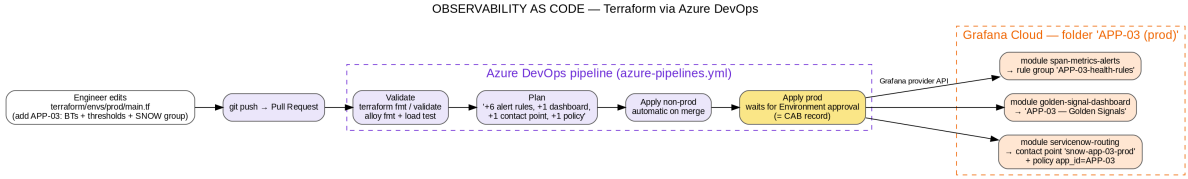
otelcol.receiver.otlp "ingress"       listens 4317/4318          otelcol.processor.transform "bt_rules"
       ▼                                                           per-app OTTL statements:
otelcol.processor.memory_limiter      protects the pod            APP-01: URI segment, header, exclude
       ▼                                                           APP-02: POJO method, JMS queue
otelcol.processor.transform "enrich"  app.id / env / namespace    APP-06: Controller/Action from URL
       ▼ (traces)                                                  APP-05: exclude /static
otelcol.processor.transform "bt_rules"  ◀──────────────────────────  global: /123 → /{id}, UUID → /{uuid}
       ▼
otelcol.processor.transform "redact"  PII + headers + SQL literals (traces AND logs)
       ▼ (traces)
otelcol.processor.tail_sampling       errors / slow / 5xx / nonprod-all / 10%
       ▼
otelcol.processor.batch "export"      ◀── metrics & logs join here
       ▼
otelcol.exporter.otlphttp "grafana_cloud"  basic auth from K8s secret
```

Why BT rules live here and not in app code: 10 apps × many BT rules = one place to maintain, no app redeploys, and the same pattern scales to 343 apps. Why redaction is here: it's the last Pacific Life-controlled hop before data leaves for SaaS.

Trade-off to know: tail sampling drops traces, and Tempo's metrics-generator only counts what arrives. So span-metrics reflect the *sampled* set unless we generate span-metrics inside the pipeline before sampling (a connector we can enable — Pacific Life decides, Q11).

---

10. Observability-as-code (Terraform + Azure DevOps)



06 terraform flow

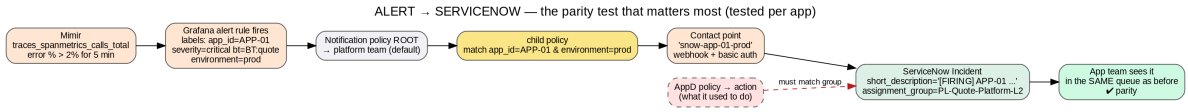
```
Engineer edits terraform/envs/prod/main.tf  (adds APP-03 with its BTs + thresholds)
       | git push → PR
       ▼
┌── Azure DevOps pipeline (azure-pipelines.yml) ──────────────────────────────┐
│ Validate: terraform fmt / validate; alloy fmt + load test                   │
│ Plan:     terraform plan → "will create 6 alert rules, 1 dashboard, 1 contact point"  │
│ Apply:    nonprod auto on merge; prod waits on ADO Environment approval (= CAB ref)   │
└─────────────────────────────────────────────────────────────────────────────┘
       | Grafana provider → Grafana Cloud API
       ▼
Grafana Cloud: folder "APP-03 (prod)" containing
  • rule group "APP-03-health-rules"      (module span-metrics-alerts)
  • dashboard "APP-03 — Golden Signals"   (module golden-signal-dashboard)
  • contact point "snow-app-03-prod"       (module servicenow-routing)
  • child notification policy app_id=APP-03 → that contact point
```

The three modules:

Module	AppD thing it replaces	Inputs	Output in Grafana
`span-metrics-alerts`	Health rules	`business_transactions` map: BT name → thresholds (error %, p95 ms, min calls/min), severity, strategy (static / ml / exception)	Alert rules on `traces_spanmetrics_*`; outputs the list of ML candidates and exceptions for the FPC
`golden-signal-dashboard`	Dashboards	service name, BT list, datasource UIDs	One dashboard: service RED, per-BT RED, BT summary table, error traces, correlated logs
`servicenow-routing`	Policy → Action	assignment group, SNOW webhook creds	Contact point (webhook) + policy matcher on `app_id`; optional always-firing "routing test" rule to prove the incident lands in the right group

---

11. Alert → ServiceNow path (the parity test that matters most)



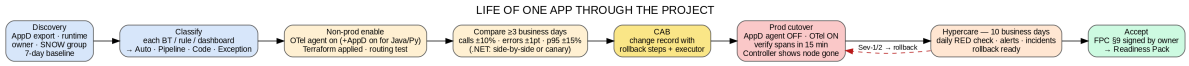
07 alert to servicenow

```
Mimir metric crosses threshold for 5m
  → Grafana alert rule fires   labels: app_id=APP-01 severity=critical bt=BT:quote environment=prod
  → Notification policy tree:  root (platform team) ——child: app_id=APP-01 & environment=prod——▶
  → Contact point "snow-app-01-prod" (webhook, basic auth)
  → ServiceNow inbound API creates Incident
       short_description = "[FIRING] APP-01 error rate > 2%"
       assignment_group  = "PL-Quote-Platform-L2"      ◀—— must equal what the AppD policy used
  → App team sees it in the same queue as before.  ✔ parity
```

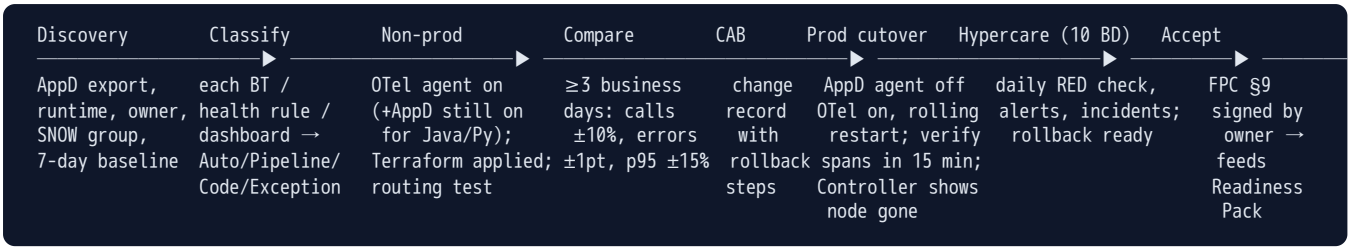
We test this **per app** with the routing-test rule, record the incident number and group in FPC §4, then disable the test rule.

---

12. Life of one app through the project



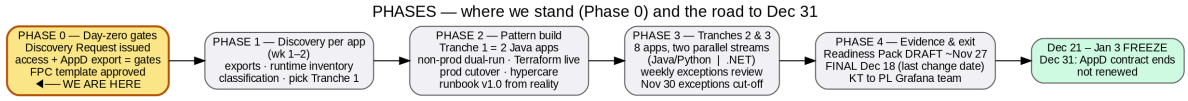
08 app lifecycle



Tranche 1 = 2 lowest-risk Java apps (we learn, write the runbook from reality). Tranche 2 & 3 = remaining 8, run as two parallel streams (Java/Python vs .NET) because the validation methods differ.

---

13. Phase map and where we stand



09 phases

Phase	What happens	Status
0 — Day-zero gates	Discovery Request issued; access + AppD export as gates; FPC template approved; proposal vs requirements reconciled	← **We are here.** Documents and reference configs drafted; nothing changed in PL systems
1 — Discovery per app	Exports, runtime inventory, classification, pick Tranche 1 apps	Next
2 — Pattern build (2 apps)	Non-prod dual-run, Terraform modules live, prod cutover, hypercare, runbook v1.0	—
3 — Tranches 2 & 3 (8 apps)	Parallel streams, weekly exceptions review, Nov 30 exceptions cut-off	—
4 — Evidence & exit	Readiness Pack draft Nov 27 / final Dec 18; KT to PL Grafana team	—

---

14. What you will actually do as L1 on this project

Daily checks during validation/hypercare

```
# Are spans arriving?          Grafana → Explore → Tempo:   { resource.service.name = "app01-quote-api" }
# Are BT metrics there?        Explore → Mimir (PromQL):
  sum by (span_name) (rate(traces_spanmetrics_calls_total{service="app01-quote-api"}[5m])) * 60
# Error % per BT:
  sum by (span_name) (rate(traces_spanmetrics_calls_total{service="app01-quote-api",status_code="STATUS_CODE_ERROR"}[5m]))
  / sum by (span_name) (rate(traces_spanmetrics_calls_total{service="app01-quote-api"}[5m])) * 100
# Logs correlated?             Explore → Loki:  {service_name="app01-quote-api"} |= "trace_id"
# Alerts firing?               Alerting → Alert rules → folder APP-01
# Pipeline healthy?            kubectl -n observability logs deploy/alloy-pipeline | grep -i error
```

Triage ladder when "no data" 1. Agent: is the process started with the OTel flag / profiler env? Agent log shows export errors? 2. Host Alloy: `curl localhost:12345/-/ready`; log shows exporter queue/retry errors → network to EKS. 3. EKS pipeline: pod logs show 401 (bad Grafana token), 429 (rate limit — ingest budget), or OTTL errors. 4. Grafana: trace present but no `traces_spanmetrics_*` → metrics-generator off (Q9). Span

names wrong → BT rule in `bt-rules.alloy`. 5. Alert fired but no incident → contact point test; policy matcher labels (`app_id`, `environment`) ; SNOW integration user.

Evidence you will collect (goes into `evidence/<APP_ID>/` and the FPC) - Screenshot of AppD BT list vs Grafana BT table (calls/min side by side). - Comparison export for the ≥3-day window. - ServiceNow incident number from the routing test. - AppD Controller node list showing "not reporting" after cutover. - Rollback rehearsal date/duration.

---

15. Repo map (what's in `/deliverables`)

```
deliverables/
├── docs/
│   ├── 00-plan-of-action.md                    the plan this guide explains
│   ├── 01-discovery-request.md                 21 questions + gates we send Pacific Life
│   ├── 02-functional-parity-checklist-template  per-app acceptance checklist (FPC)
│   ├── 03-migration-pattern-runbook.md         step-by-step per app, per runtime
│   ├── 04-decommission-readiness-pack-outline  the final evidence pack structure
│   ├── 05-app-register.md                      10 apps, tranches, validation method
│   ├── 06-exceptions-register.md               weekly exceptions review log
│   ├── 07-rda-review-additions.md              added risks/dependencies/assumptions
│   └── 08-l1-engineer-guide.md                 this document
├── alloy/
│   ├── host/config.alloy                       host collector (reference)
│   └── eks/config.alloy, bt-rules.alloy, values.yaml   regional pipeline + Helm values
├── terraform/
│   ├── modules/span-metrics-alerts             health rules
│   ├── modules/golden-signal-dashboard         dashboards
│   ├── modules/servicenow-routing              contact point + policy + routing test
│   └── envs/nonprod/main.tf                    example: APP-01, APP-02 wired up
├── agents/java.md, dotnet.md, python.md        exact flags/env vars/commands per runtime
├── azure-pipelines.yml                         CI/CD: validate → plan → apply
└── docx/                                       Word versions of every doc
```

---

16. Ten things to remember

1. We are replacing the **agent inside the app**, not the apps and not Grafana — Grafana Cloud already exists.
2. **BT = span-metrics series** (`service` + `span_name`). If `traces_spanmetrics_*` isn't in Mimir, nothing else works.
3. **.NET can't dual-run.** Validate side-by-side or canary; rollback is a redeploy.
4. **Parity is defined by the FPC**, not by feelings. Get it approved before touching Tranche 1.
5. **An alert is parity only if it lands in the same ServiceNow group.** Test per app.
6. **Everything is code** in Azure DevOps — no clicking in Grafana UI for anything that must survive.
7. **PII is scrubbed in the EKS pipeline**, the last Pacific Life-controlled hop.
8. **Tail sampling** saves money but changes what metrics count — decision for Pacific Life.
9. **AppD Controller stays as fallback until Dec 31** — keep access, use it for the baseline and the agent-removal proof.
10. **Dec 18 is the last change date** (freeze Dec 21 – Jan 3); draft evidence pack by Nov 27 so the non-renewal decision is informed.